

Technical Appendix

Public Attitudes Towards AI Across Application Domains: A Triangulation of Survey Data and Reddit Discussions

Contents

1. Introduction	3
2. Environment	3
3. Data Access	3
4. Notebooks overview	4
5. Survey Analysis Notebook	5
5.1 Preprocessing and imputation	5
5.2 Descriptive analysis	6
5.3 Latent class analysis	7
5.4 Multinomial logistic regression	9
5.5 Robustness checks	10
6. Reddit Analysis Notebook	13
6.1 Corpus retrieval	13
6.2 Corpus filter	15
6.3 Sentiment scoring	17
6.4 Cross-modal alignment	18
7. Anonymisation Notebook	13

1. Introduction

This appendix describes the procedures needed to reproduce result in the report. Three notebooks and the aggregated statistical outputs used to generate the report's figures is available at the repository link above. The appendix does not contain the raw survey file or the Reddit corpus.

2. Environment

The analysis was carried out in Python 3.11 running in Jupyter Notebook on macOS. The three packages that materially affect the results are stepmix (3.0.0) for the primary latent class analysis and the BCH three-step correction, statsmodels (0.14) for the multinomial logistic regression and scikit-learn (1.4) for the IterativeImputer in the missing data step. Other packages used include pandas, numpy, scipy, matplotlib, vaderSentiment, nltk (with the punkt_tab tokeniser) and requests.

3. Data Access

Two datasets are used. None of them is stored in the repository, and both need to be obtained separately before the notebooks can be run.

The survey data are from the Ada Lovelace Institute and the Alan Turing Institute's Wave 2 2024/25: How do people feel about AI?. They are hosted publicly on the Institute's GitHub repository at <https://github.com/AdaLovelaceInstitute/wave-2---how-do-people-feel-about-AI/blob/main/README.md>, where both the raw dataset and the technical report can be downloaded. The analysis uses the CSV version of the raw file, and

Latin-1 encoding is needed when reading it. Specific variables used are the sixteen attitude items and the six demographic covariates used as predictors.

The Reddit corpus is omitted from the repository to comply with the project's ethics requirements, even after anonymisation (author names are replaced with a placeholder, URLs are dropped, and post IDs are hashed, while titles and bodies are kept for sentiment scoring), the texts may still be traced via search engines. Anyone wanting to work with the exact corpus used in the analysis can request it by email at im10yeah@gmail.com. Alternatively, the corpus can be reconstructed by running the retrieval code in the Reddit notebook against the current Arctic Shift snapshot, though the results will not be the same because the archive continues to be updated.

4. Notebooks overview

The analysis is divided into three notebooks.

`Subm_Ada analysis.ipynb` includes things based on the survey data. It loads the raw file, cleans and imputes the sixteen attitude items, computes the domain-level descriptives that address RQ1, fits the latent class solutions and selects $k=3$, runs the multinomial logistic regression of class membership on the six demographic predictors, and carries out the two robustness checks (split-sample validation and BCH three-step re-estimation). Section 5 of this appendix records this notebook.

`Subm_Reddit analysis.ipynb` handles all Reddit data, retrieves posts from the Arctic Shift API, applies the four-tier precision filter, removes some posts, preprocesses the text for VADER, scores sentiment at the sentence level, computes the per-domain summary statistics and conducts the cross-modal

alignment against the survey attitudes. See section 6 for the full.

submission archive anonymisation.ipynb applies SHA-256 hashing to post IDs, replaces author names with a placeholder and drops URL fields from the Reddit files. Section 7 documents this notebook.

5. Survey Analysis Notebook (Subm_Ada analysis.ipynb)

This section is organized into five sub-sections that match the five analytical stages of the notebook: preprocessing and imputation, descriptive analysis, latent class analysis, multinomial logistic regression and robustness checks.

5.1 Preprocessing and imputation

The notebook began with reading the raw CSV file with `pandas.read_csv` and `encoding= 'latin-1'`. The raw file has 3513 rows and 342 columns.

Only 25 of these columns are kept for analysis: the panel identifier `panelid_2`, the survey weight `Nov24_Weight`, the sixteen attitude items grouped as (benefit, concern) pairs across the eight application domains, and seven demographic variables (`Cur_Sex`, `Cur_AgeCat`, `Cur_HEdQual`, `Cur_GOR`, `Cur_PartyID5`, `Cur_RClassGp`, `Cur_EconAct5`). Of these, six are subsequently used as predictors of class membership (all except `Cur_GOR`); the seventh is retained in the working dataset for completeness but is not used in any analytical step.

Missing values need to be handled in two separate process because the raw file uses two different conventions for missingness. The four negative codes -1, -8, -9 and -11 mark skipped or unanswered questions and are recoded to NaN

across all attitude and demographic variables in a single replace call. The value 5 on the attitude items is the “Don’t know” option, which is different from a skip but analytically equivalent to a missing value under the study’s design, so it is also recoded to NaN, but only on the sixteen attitude items and not on the demographic variables where 5 is a valid category.

After the missing codes are handled, the sixteen attitude items are reverse-coded so that higher scores represent stronger endorsement of the underlying construct. In the raw file, 1 corresponds to the strongest position on item’s own scale (“very beneficial” for benefit items, “very concerned” for concern items) and 4 to the weakest. The recode map {1: 4, 2: 3, 3: 2, 4: 1} reverses this, so under the recoded scale a high score on a benefit item means the respondent think the application is more beneficial, and a high score on a concern item means they are more concerned. Reverse coding is applied by mapping each of the sixteen original items to a new column with the suffix `_r`, and it is the `_r` variables that are used in every subsequent analytical step. Listwise deletion on all sixteen `_r` items retains only 1740 of the 3513 respondents. Due to the differences suggest the missingness is not completely at random, the analysis proceeds with multivariate imputation.

Imputation use `IterativeImputer` from `sklearn.experimental` with the following: `max_iter=50`, `random_state=42`, `min_value=1`, `max_value=4`, and `tol=1e-3`. The value bounds constrain imputed values to the range of the Likert scale, and the resulting continuous imputations are then rounded to the nearest integer with `.round().astype(int)` to keep the ordinal structure of the responses. Only the sixteen `_r` variables enter the imputation model. Demographic variables are not imputed because their missingness is handled separately by casewise deletion at the multinomial regression stage.

5.2 Descriptive analysis

The descriptive step calculate the mean, standard deviation and valid observation count for each of the sixteen reverse-coded attitude items.

An important point about which dataset is used: descriptive statistics are computed on `df_subset` deliberately , which is the pre-imputation dataframe where missing values remain as `NaN` . The imputation carried out in 5.1 is needed to give the latent class analysis a complete 3513 rows input, but the descriptive means reported for RQ1 are more transparent if they are computed on observed responses only. As a result, the valid-n for each item in “Figure perceives benefit vs. Concern by AI application domain”(Figure 1 of the report) ranges from 2768 (BenChatbot) to 3420 (ConFR). From 5.3 onwards, the notebook switches to `df_lca`, the fully imputed dataset.

The computation loops over the eight domains. For each domain, the two `_r` items are pulled from `df_subset` and their means, standard deviations and valid counts are stored in a summary dataframe, sorted by concern mean in descending order. Figure 1 is a dumbbell plot generated with `matplotlib`, with error bars representing the standard error of the mean ($\text{std}/\sqrt{n}$). Domains are ordered by ascending benefit mean using `sort_values`.

5.3 Latent class analysis

The LCA step takes the imputed dataframe `df_lca`, fits categorical latent class solutions for $k=2$ through $k=8$, selects $k=3$ as the final solution and produce the class assignments and posterior probabilities. The input matrix `X_lca` is `df_lca[recoded_vars].values`, a 3513×16 integer array on the 1 to 4 Likert scale.

All models are fitted with StepMix from the `stepmix.stepmix` package (3.0.0).

The measurement= 'categorical' argument tells StepMix to treat each of the sixteen items as a four-category variable, and the package fits a single model per call. The initial class enumeration loop runs 10 random seeds for each k from 2 to 6, with n_init=1, max_iter=500, and progress_bar=False on each individual fit. The best BIC across the 10 seeds is retained. Once the initial range was explored, the loop was extended to k=7 and k=8 with the same procedure. This gives seven candidate solutions. Normalized entropy is computed manually. For a k-class solution with posterior probability matrix P of

shape (n, k), the formula used is

$$1 + \frac{\sum_{i=1}^n \sum_{j=1}^k P_{ij} \ln(P_{ij})}{n \ln(k)}.$$

The four criteria that inform the final choice of k are BIC, AIC, normalized entropy and interpretability of the class profiles. Once k=3 is chosen, the final model is refitted with n_seeds=20 rather than 10 to guard more strongly against local optima. This is done through a helper function, fit_best_model(X, k, n_seeds), which is called with n_seeds=20 for both k=3 and k=5

The class labels “AI Enthusiasts”, “Cautious Moderates” and “AI Sceptics” are not produced by StepMix. StepMix returns raw class indices 0, 1, 2. The labels are assigned manually after inspecting the fitted profile means, based on which class is most favourable across all sixteen items (Enthusiasts), which most unfavourable (Sceptics), and which sits between the two (Moderates). In the specific k=3 model produced, raw class 0 corresponds to Enthusiasts, raw class 1 to Moderates, and raw class 2 to Sceptics, and the code adds 1 to shift the labels to 1-indexed before writing them into df_lca['LCA_class'] and df_lca['LCA_class_label']. This 1-indexed assignment is the input to 5.4, and the label-to-index mapping matters again in 5.5 for the split-sample and BCH alignment steps. df_lca['LCA_class'], an integer column taking values 1, 2 or 3 for each respondent's modal class, and the three columns prob_class1, prob_class2, prob_class3, are which hold the full posterior distribution over

classes for each respondent.

5.4 Multinomial logistic regression

The regression step models modal class membership on the six demographic covariates. The input is `df_lca`, and the output is a fitted multinomial logistic regression (MNLogit) result object plus the odds ratios and confidence intervals. The implementation use statsmodels's MNLogit. The regression sample is built by taking the six demographic variables with the `LCA_class` column, replacing the four negative missing codes (-1, -8, -9, -11) with NaN, and dropping any row with a missing value on any covariate. Only the demographic covariates are subject to casewise deletion at this stage.

What is important about implementation in this section is the class recoding done before the model is fitted, MNLogit treats the numerically smallest value in the outcome variable as the reference class. The study use Cautious Moderates as the reference, but Moderates is originally coded as class 2, with Enthusiasts as 1 and Sceptics as 3. Fitting the model on the raw class labels would wrongly set Enthusiasts as the reference, which would flip the interpretation of every coefficient. It was fixed with the mapping `class_recode_for_ref = {2: 0, 1: 1, 3: 2}`, applied to `df_reg['LCA_class']` before the model is fitted. Under this mapping, the recoded outcome takes value 0 for Moderates (reference), 1 for Enthusiasts and 2 for Sceptics.

The design matrix is built with `pd.get_dummies(df_reg[demo_vars_used].astype(str), drop_first=True, dtype=float)`. The variables are cast to strings before dummy-coding. The `drop_first=True` argument makes the first category of each variable the reference, because the variables are sorted by numeric codes, the reference categories come out as Female, 18-29, Degree, Conservative, Managerial and

professional, and Full-time education. An intercept is then added with `sm.add_constant`. The model is fitted with `sm.MNLogit(y, X_reg).fit(method='bfgs', maxiter=500, disp=False)`. BFGS is specified explicitly.

The `result.params` object is a 24×2 DataFrame, the 24 rows are the intercept plus the 23 dummy columns, in the order of they appear in `X_reg`. The two columns are indexed as integers 0 and 1, where column 0 holds the log-odds contrast for the recoded class 1 (Enthusiasts vs. Moderates) and column 1 holds the contrast for the recoded class 2 (Sceptics vs. Moderates). `result.conf_int()` returns a 48-row DataFrame with a MultiIndex whose outer level use the string values '1' and '2' (not integers) to identify the two contrasts. This shows up in the plotting code as `conf.loc[('1', plot_vars), 'lower']` and similar, getting the indexing type wrong here silently returns an empty selection.

Odds ratios are obtained by applying `np.exp` to the log-odds, and confidence intervals are computed the same way on the lower and upper columns of `conf_int()`. The plotting code loops over `panel_configs` which pairs each panel with the correct `param_col` and `conf_key` combination.

5.5 Robustness checks

This section runs two independent checks on the $k=3$ solution and its covariate associations. The first splits the imputed sample into two halves and refits the LCA on each; second re-estimates the class-covariate relationships using StepMix's built-in three-step estimator with the BCH correction.

5.5.1 Split-sample validation

The split is produced with `train_test_split` from `sklearn.model_selection`, called with `test_size=0.5` and `random_state=42`. The seed is fixed so reruns produce the same partition, the result is two subsamples of size 1756 and 1757. Each subsample is passed to the same `fit_best_model` helper with `k=3` and `n_seeds=10`. As a first check, AIC, BIC, and normalized entropy are computed for both subsample solutions using the same manual entropy formula from 5.3.

Because raw class indices are arbitrary across fits, the subsample class labels are aligned to the full-sample labels. Before any comparison can be made, the subsample class labels have to be aligned to the full-sample labels. So for the alignment, the function `get_profiles(model, X, recoded_vars)` computes each class's mean profile across the 16 attitude items and returns as a (3, 16) matrix indexed by raw class number. Then `align_to_reference(profiles_source, profiles_ref)` computes the pairwise distance between the subsample profile matrix and the full-sample profile matrix using `scipy.spatial.distance.cdist`, which defaults to Euclidean distance, and passes the resulting 3×3 distance matrix to `scipy.optimize.linear_sum_assignment`. This is the Hungarian algorithm and it returns the assignment of subsample rows to full-sample rows that minimizes the total distance. The returned dictionary maps each subsample raw class index to a full-sample class index.

Once the mappings `map_A` and `map_B` are available, the subsample profile matrices are reordered so that row `j` corresponds to the same class across all three solutions, and the subsample class labels are relabelled the same way. Profile similarity is quantified with Pearson correlations computed on the 16-dimensional class mean vectors, giving three correlations each class: full vs. A, full vs. B, and A vs. B. Average posterior probabilities are reported per aligned class within each subsample, using the same threshold logic from 5.3. "Figure Split-sample robustness of the `k=3` latent class solution" is produced by looping over the aligned class index and plotting the full, A, and B profile

vectors on the same axes. The `df_lca` dataframe remains not be modified.

5.5.2 BCH three-step re-estimation

The BCH check refit the class-to-covariate model using StepMix's built-in three-step estimator, applying the BCH correction to account for classification uncertainty, it is compared against the two-step MNLogit result to see whether the reported covariate effects are sensitive to the way class membership is treated in the second-stage regression.

The input matrices are built on the same 3412-row subset to ensure comparability. The mask `mask_reg = df_lca.index.isin(df_reg.index)` select respondents who survived the casewise deletion of demographic missings, and `X_lca_reg` is the corresponding 3412×16 attitude matrix cast to integer. The covariate matrix `Z_cov` is built from `df_reg[demo_vars_used]` using `pd.get_dummies(..., drop_first=True, dtype=float)`, giving a 3412×23 array. Fitting is done through a `fit_best_bch` helper, which loops over ten random seeds, constructs a StepMix model `n_components=3, n_steps=3, measurement='categorical', structural='covariate', assignment='modal', correction='BCH', max_iter=500, structural_params={'method': 'newton-raphson', 'max_iter': 200, 'intercept': True}`, and picks the model with the lowest measurement BIC.

The alignment maps BCH raw classes to the main model's class indices. Profile means are computed for both solutions using `get_class_profiles`, the main model labels are from `df_lca` and changed from 1-indexed back to 0-indexed with `labels_main_reg = df_lca.loc[mask_reg, 'LCA_class'].values - 1`, and `align_to_reference` runs the Hungarian algorithm on the pairwise Euclidean distances between the two profile matrices. The resulting `map_bch` dictionary translate BCH raw class indices to main model class indices and its

inverse `inv_map_bch` is used a few lines later to reorder the coefficient matrix.

The structural coefficients are extracted through `model_bch.get_parameters()['structural']['beta']`, the most important technical point is that StepMix's covariate model use a softmax parameterization, meaning each row of beta gives the log-odds of that class relative to a symmetric average. To make the two sets of coefficients comparable, the BCH beta rows have to be differenced. This dose in two lines: `bch_contrast_enth = beta_bch[0] - beta_bch[1]` and `bch_contrast_scep = beta_bch[2] - beta_bch[1]`, where the row indices refer to the aligned class ordering (Enthusiasts, Moderates, Sceptics).

Before comparing coefficients, one more sanity check confirms the covariate column order is identical between the two-step and BCH designs. The two-step `X_reg.columns` starts with 'const' followed by the 23 dummy columns from `pd.get_dummies`, and the BCH design has an intercept added by StepMix from `Z_cov_names`. Agreement between the two sets of estimates is quantified on the 23 slope coefficients excluding the intercept. The metrics are Pearson r , Spearman ρ , mean absolute difference, RMSD, and the proportion of coefficients where the two methods agree on the sign. These are computed separately for the Enthusiasts and Sceptics contrasts.

6. Reddit Analysis Notebook (Subm_Reddit analysi.ipynb)

This section covers the Reddit corpus construction and analysis, follows the retrieval from the Arctic Shift API, filtering the raw posts, sentiment scoring and cross-modal alignment against the survey attitudes.

6.1 Corpus retrieval

Posts are retrieved from the Arctic Shift archive at <https://arctic-shift.photon-reddit.com>, which mirrors Reddit data and exposes a REST endpoint at /api/posts/search. Each call takes a subreddit name, a query string, an after and before timestamp, a page size limit, a sort direction and a comma-separated list of fields to return. The study requests 10 fields: id, subreddit, author, created_utc, title, selftext, score, num_comments, url and link_flair_text. The time window is fixed from 2024-06-01 to 2025-06-01, spanning and extending beyond the Ada Lovelace survey fieldwork period.

SUBREDDITS is a list of six subreddit names: four UK-focused (unitedkingdom, CasualUK, AskUK, UKPolitics) and two general technology-focused (technology, artificial). DOMAIN_QUERIES is a dictionary mapping each of the eight AI application domains to its primary query set, with phrase queries wrapped in double quotes to enforce exact-match search on Arctic Shift. SUPPLEMENTARY_QUERIES and ROBOCARE_FINAL_QUERIES hold the additional keywords added after the primary retrieval for the three low-yield domains (Loan, Chatbot, RoboCare).

Use two versions of the page-fetching function. fetch_page is used for the initial pass and handles 429 rate limiting by waiting based on the X-RateLimit-Reset header and retrying recursively. All reruns use fetch_page_v2, which handles both 429 and 422; a 422 triggers a fixed 15-second wait and up to three retries before giving up. fetch_all handles pagination because the API returns at most 100 posts each call and the twelve-month window can hold thousands of posts for popular queries, the function loops with sort= 'asc' and advances the cursor after each batch to the created_utc of the last post returned. Duplicate posts at the page boundary are removed by keeping a seen_ids set and filtering each new batch against it. After the primary pass finishes, combos with status == 'error' in the retrieval log

was identified, cached files were deleted and the process was rerun with the v2 fetching functions. Once that pass completes with only a handful of remaining errors, supplementary keyword rounds are added for domains that returned very few UK-relevant posts. Each supplementary round runs the same fetch loop against all six subreddits, caching to the same directory structure.

6.2 Corpus filter

The filtering stage takes the 2992 raw records, deduplicates them and applies four layers of precision filtering plus a news-digest exclusion, producing the final 181 post sentiment analysis corpus. The notebook iterates over all posts and inserts each into a dictionary keyed by post id. Duplicates arising from the same post being returned under multiple query or subreddit combinations are dropped at this point by keeping only the first occurrence of each id. After deduplication, 2818 unique posts enter the filtering pipeline.

The four filter layers are applied in a single loop, in order. If a post fails one layer, it is dropped without evaluating subsequent layers, and the drop reason is recorded in a `drop_counts` counter for the final tally and the order matters. The first layer removes deleted posts, the post is treated as deleted if both its title and body are in the set `{"[deleted]", "[removed]", "[deleted by user]", ""}`. Length checking is the second layer. A post has to have a title of at least 15 characters, and if its body is empty the title alone has to reach 30 characters; the combined title with body also has to reach 30 characters overall. The third layer handles UK focus. The `UK_PATTERN` regex matches any of the standard UK-referential terms; the four UK-focused subreddits pass this layer automatically; the two general-purpose subreddits (technology, artificial) only pass if the combined post text matches the UK pattern at least once. This filters out 2094 posts from the general subreddits that turned out to be non-UK content.

The fourth layer is the domain-specific matcher, defined by the DOMAIN_RULES dictionary. Each of the eight domains has a core pattern list capturing the domain-defining terms (like facial recognition, universal credit, ChatGPT) and a context pattern list capturing terms that establish the post is actually about that domain in an AI context (like police, algorithm, mental health). A rule counts as satisfied when at least one core pattern and one context pattern matches. Except for two special cases, the LLM and Car rules have context: None, meaning core matches alone are sufficient because their core terms are already unambiguous. Cancer takes the opposite direction: it uses an extra lambda function requires the co-occurrence of two distinct term groups, one AI-related (AI, algorithm, machine learning, neural network, deep learning, or the like) and one scanning-related (scan, screening, diagnos, detection, imaging, radiolog, mammogra, MRI, CT scan), because the “cancer” alone would produce far too many false positives. Every post is evaluated against all eight domain rules, not just against the domain it was originally fetched under, since keyword overlap between domains can pull a post into more than one domain’s retrieval bucket. When a post satisfies multiple domain rules, the tie is broken by choosing the domain with the highest n_core count. Ties on this count are resolved by dropping the post entirely. Of the 188 posts retained, most satisfied exactly one domain rule; however, two posts originally fetched under Chatbot were reassigned to LLM because they did not pass the Chatbot rule but passed the LLM. Three additional posts were discarded due to a tied n_core count across multiple domains.

A final exclusion step removes news-digest posts before the corpus enters sentiment scoring. The r/artificial subreddit hosts a recurring “One-Minute Daily AI News” post that concatenate several unrelated headlines into a single submission, and retaining these would let external-headline valence contaminate the user-discussion sentiment estimates. The notebook uses two

independent detectors as a cross-check: one is an exact regex match against the title pattern “One-Minute Daily AI News”; the other is a structural heuristic that flags posts whose title contains a date pattern and whose body contains at least three enumerated items. The rules for both detectors were fixed before the code was run so that the exclusion cannot be accused of being tuned to the retrieved data.

6.3 Sentiment scoring

Sentiment scoring convert the 181 filtered posts into per-post and per-domain sentiment estimates using VADER (vaderSentiment 3.3.2). The step has three parts: text preprocessing tailored to VADER, sentence-level scoring with post-level aggregation and bootstrap confidence intervals for the domain means.

Preprocessing is in a single function, `preprocess_for_vader`. Letter case is a valence-modifying feature in VADER’s rule: “GREAT” scores higher than “great”; exclamation marks and question marks also modify intensity; punctuation, emoticons and emoji are similarly meaningful. The preprocessing therefore keeps all of these and removes only content that carries no sentiment signal or would confuse the tokeniser. URLs, markdown links, subreddit mentions, user mentions, zero-width characters and repeated whitespace are stripped. HTML entities are unescaped. Preprocessing is applied to the `combined_text` column, writing the result into `text_for_vader`. Four fields are added at the same time: `vader_char_len`, `vader_word_len`, `orig_char_len` (used to derive the removal percentage), and `char_removed_pct`

Sentence-level scoring uses NLTK’s Punkt tokeniser. The function `score_post_sentencewise` splits the preprocessed text with `sent_tokenize`,

runs VADER on each resulting sentence, and aggregates the sentence-level compound scores into a post-level record. The number of sentences, the mean, median, standard deviation of the sentence-level compound scores, and the VADER's conventional thresholds are stored. Scoring sentence by sentence and then averaging preserves the strength of individual sentiment-carrying sentences and produces more stable domain-level estimates. The whole-post compound score is also computed and stored in `compound_wholepost` as a sensitivity check.

Bootstrap confidence intervals for the per-domain means are computed with `bootstrap_ci`. For a given domain's vector of post-level compound means, the function uses `np.random.default_rng(42)`, computes the sample mean of each resample and returns the 2.5 and 97.5 percentiles. If a domain has fewer than 2 valid observations, the function returns `(NaN, NaN)`. The domain summary is assembled by looping over the eight domains in a fixed order and collecting the mean, median, standard deviation, bootstrap CI bounds, whole-post mean and the percentage of posts that are net positive or net negative under the 0.05 threshold.

6.4 Cross-modal alignment

The alignment step compares the survey attitudes against the Reddit sentiment estimates at population-level and class-level. The survey concern means are hard-coded: population means in `survey_concern`, and class-specific profiles in `concern_by_source`.

The Reddit side is loaded from `_domain_sentiment_summary.csv`. One of them has to be sign-flipped for the rank comparison to be interpretable because higher survey concern means more unease but higher Reddit compound sentiment means more positive. The Reddit side with `reddit_neg =`

`{d: -v for d, v in reddit_sent.items() if not pd.isna(v)}` was flipped, so that under both axes higher values point in the same direction. Domains with missing Reddit means are filtered out. Domain inclusion follows explicit rules recorded in an inclusion function that tags each domain with its status. Chatbot is excluded because $n=0$; Cancer is excluded because VADER assigns a strong negative valence to the token “cancer” itself, which contaminates the two Cancer posts and produces domain-mean estimates that reflect lexicon bias; RoboCare is excluded because its single retrieved post was found to be a semantic false positive discussing human care professionals. Domains with n between 4 and 9 (Loan and Car) are flagged as sensitivity-only by the inclusion rule. In practice, Car ($n=9$) is retained in the primary comparison set because it sits very near the threshold and the report explicitly treats it as part of the primary analysis, so primary set comprises FR, LLM, WB, and Car; the sensitivity set adds Loan.

Population-level alignment uses Spearman rank correlation. Because the number of domains is very small, the asymptotic p-value from `scipy.stats.spearmanr` is unreliable, so the notebook computes an exact two-sided p-value by computing Spearman ρ under each permutation, and counting the fraction whose absolute value equals or exceeds the observed ρ . This is done in `spearman_with_exact_p`, the asymptotic p from `scipy` is also as a reference. Class-level alignment generalise the same permutation framework. The dictionary `concern_by_source` holds the population concern means and the three class-specific profiles (Enthusiasts, Moderates, Sceptics) vectors. For each source, both Pearson r and Spearman ρ are computed against the Reddit negative-sentiment vector, each with an exact permutation p . The function `exact_permutation_p` take a `stat` argument that switch between Pearson and Spearman by choosing the corresponding `scipy` function inside the permutation loop.

7. Anonymisation Notebook (submission archive anonymisation.ipynb)

This notebook is a standalone script run after all analysis was done to produce a version that could meet the project's ethics approval. The anonymised archive is not included in the repository, but the notebook is included so that the anonymisation procedure is fully transparent and can be verified.

Each Reddit record apply following modifications, author usernames are replaced with the placeholder string [anonymised]; URL fields are dropped entirely; post ids are replaced with the first 12 hexadecimal characters of their SHA-256 hash; content fields including title, body and any derived text columns are kept unchanged, because sentiment scoring depends on them. Raw JSON files under reddit_raw/ are read one at a time, each post dict is passed through anonymise_post_dict, and the modified list is written to the mirror path under reddit_raw_submit/. Cleaned CSV files under reddit_clean/ are read into pandas, passed through anonymise_dataframe which drops the author and url columns and apply the hash to the id column, then written to reddit_clean_submit/.

Applying some verificatio steps. Anonymised raw archive is walked until a non-empty JSON file is found, and that file is loaded and checked field by field; the original and anonymised versions of _all_posts_scored.csv are loaded side by side and the twelve columns used in downstream analysis (including compound_mean, compound_median, compound_std, the three sentence-percentage fields, compound_wholepost, n_sentences, assigned_domain, subreddit, combined_text, and text_for_vader) are compared with pd.Series.equals; the hash function is applied twice to the same input to confirm the mapping is deterministic.